

# Linearity of Meaning<sup>1</sup>

Junichi Iida

## 1. Types

I propose a terminological revision of the formal semantic framework described in *Semantics* (Iida 2026). The proposed changes are for clarity and no substantive modifications of the formal system are made at this time. I believe the revision better reflects the essence of Convention 4.1 (p. resident), reproduced below, than the original terminology:

“  
Convention 4.1:

i)  $\langle X \rangle$   $\langle_R \rangle$

$$\langle_R X \rangle = \left\langle \frac{{}_R A \cdot {}_R X}{{}_R B \cdot {}_R Y} \right\rangle \neq \langle X \rangle$$

*A role takes an immediately preceding serial to form a serial with a role. Serials with and without a role are considered different types. Only the former can be taken by a parallelizer.*

(*Semantics*, p. resident)

In Convention 4.1, the angle brackets are used as a function that returns the type of what is enclosed in it. The key claims of the convention are: a) When  $\langle X \rangle$  and  $\langle_R \rangle$  are placed adjacently in this order, they can merge, b) When  $\langle X \rangle$  and  $\langle_R \rangle$  merge, they become  $\langle_R X \rangle$ , c) a *parallelizer* (explained below) can take, i.e., merge leftward with, the type of objects  ${}_R X$  and  $\frac{{}_R A \cdot {}_R X}{{}_R B \cdot {}_R Y}$ , but not  $X$ .

In the original terminology,  $X$ ,  ${}_R X$ , and  $\frac{{}_R A \cdot {}_R X}{{}_R B \cdot {}_R Y}$  were indiscriminately called *serials*. But as Convention 4.1 defines,  $X$  is distinguished from the other two. And I feel that the terminology will be more comfortable with distinct names for  ${}_R X$  and  $\frac{{}_R A \cdot {}_R X}{{}_R B \cdot {}_R Y}$ . Let us call  $X$  a *bud*,  ${}_R X$  a *flower*, and  $\frac{{}_R A \cdot {}_R X}{{}_R B \cdot {}_R Y}$  a *cyme*. *Serial* is used as a cover term for the flower and the cyme but it does not refer to a bud.

The parallelizer was originally defined as follows. We continue to use this definition of parallelizer except that we adopt Convention 4.1.

---

<sup>1</sup> An official document of Kyouro/教育ローマ字; Version 7.2.0, updated to June 2026. (Original: Version 7.2.0, released in June 2026.)

“A parallelizer has a symbol on the left side followed by a colon, which is then followed by a vinculum with a symbol on each side. An example of a parallelizer is  $X: \frac{X', X''}{X}$ . This type is denoted by the symbol

$$\left\langle X: \frac{Y}{Z} \right\rangle.$$

(Semantics, p. gods)

A parallelizer takes a serial and generates a *parallel*, which is defined as follows:

“A parallel has a vinculum and a symbol on each side. An example of a parallel is  $\frac{\text{is}^{\text{cute}}}{\neg \text{is}^{\text{cute}}}$ . This type is

denoted by the symbol  $\left\langle \frac{X}{Y} \right\rangle.$

(Semantics, p. gods)

A serial may merge with a *mood*. The mood is a node type defined in pp. elegance-eloquence of *Semantics*. It appears at the end of a clause structure, and when it merges, the root node (which is also a node type) of the clause is obtained.

Node types may be represented by special symbols. A summary of node types in the original and revised terminologies and their corresponding symbols are given below:

Figure 1.1: Summary of the original and revised terminologies and symbols for types

| ORIGINAL TERMS: | REVISED TERMS: | SYMBOLS: |
|-----------------|----------------|----------|
| Serial          | Bud            | $B$      |
| Role            | Role           | $R$      |
| Serial          | Serial         | $S$      |
| Parallel        | Parallel       | $P$      |
| Parallelizer    | Parallelizer   | $[S: P]$ |
| Mood            | Mood           | $M$      |
| Root node       | Root node      | ROOT     |

Buds, roles, serials, parallels, parallelizers, and moods undergo merge to recursively generate new nodes according to Rule 1.1. Merge replaces the input nodes with the resulting node in situ. For a structure  $[B, R, [S: P]]$ , if  $B$  and  $R$  merge to generate  $S$ ,  $S$  and  $[S: P]$  will then be adjacent. None of the rules are new. The rule  $P, P: S$  is given in p. gods of *Semantics*, stated before adopting Convention 4.1.  $X, Y$  denotes a pair of  $X$  and  $Y$  given in this order. A colon indicates a mapping relation.  $X: Y$  is a function  $f(X) = Y$ .

Rule 1.1: Type-based merge constraints of clauses

Within a clause, only pairs of adjacent nodes satisfying one of the following input conditions may merge to generate a node of the output type.

- $B, R: S$
- $S, [S: P] : P$
- $P, P: S$
- $S, M: \text{ROOT}$

## 2. Linearity

I propose formal generalizations of attested semantic base structures of Tokyo Japanese. While I expect that the present generalizations reflect some aspects of the nature of language, no evidence concerning the extent to which the generalizations hold is presented at this time.

All clauses described in *Semantics* are in the pattern  $\left[ [B_R, [S: P]]^i, B_R, [S: P]^i, M \right]$ , that is, a sequence  $[B_R, [S: P]]$  repeated  $i$  times, followed by a sequence  $[B_R]$ , followed by  $[S: P]$  repeated  $i$  times, which is then concluded by  $M$ . Let us call this type of clause the *rhipidial* clause.

Definition 2.1: Rhipidial clause

A clause whose semantic base is a linear structure  $\left[ [B_R, [S: P]]^i, B_R, [S: P]^i, M \right]$ , where a superscripted variable indicates the number of repetition, is a rhipidial clause.

All rhipidial clauses conform to Rule 1.1,<sup>2</sup> but the inverse is not true.<sup>3</sup> It is not yet known whether a non-rhipidial clause can appear in a sentence of Tokyo Japanese.

A sentence may consist of multiple clauses. The root node of a relative clause is obtained with a special mood called the index mood merging with a serial. It then merges with a bud to generate a bud which becomes a part of the outer clause. The rules of the relative root are given in *Semantics* as Convention 4.2:

<sup>2</sup> If the base structure generates  $[S, M]$  for  $i = n$ , for  $i = n + 1$  it generates  $[P, S, [S: P], M]$ , which generates  $[S, M]$ . For  $i = 0$ , the base structure is  $[B_R, M]$ , which generates  $[S, M]$ .

<sup>3</sup>  $B_R [S: P] B_R [S: P] [S: P] B_R [S: P] B_R [S: P] [S: P] M \rightarrow PP [S: P] PP [S: P] M \rightarrow PPM \rightarrow \text{ROOT}$

“

Convention 4.2:

$$i) \frac{{}_R\text{index}_n \cdot X}{0} \quad i \quad X_n$$

$$\left[ {}_R\text{index}_n \cdot X \right] i$$

$$X_n$$

$$ii) \frac{{}_R\text{index}_n \cdot X}{0} \quad i \quad X_m$$

$$\left[ {}_R\text{index}_n \cdot X \right] i$$

$$*$$

(where  $n \neq m$ )

$$iii) \frac{{}_R A_n \cdot X}{0} \quad i$$

\*

(where  $A \neq \text{index}$ )

(Semantics, p. resident)

where  $i$  is a mood,  $X$ ,  $X_n$ , and  $X_m$  are buds,  ${}_R\text{index}_n$  and  ${}_R A_n$  are flowers,  $\frac{{}_R\text{index}_n \cdot X}{0}$  and  $\frac{{}_R A_n \cdot X}{0}$  are cymes, and  $\left[ {}_R\text{index}_n \cdot X \right] i$  is a root node. Asterisks indicating failure of merge are placed where the resulting nodes would be. While the convention is written in terms of specific mood and buds, here we are only interested in types. From Rule 1.1 and Convention 4.2, the following unified rule is obtained:

Rule 2.1: Type-based merge constraints of sentences

Within a sentence, only pairs of adjacent nodes satisfying one of the following input conditions may merge to generate a node of the output type.

- $B, {}_R : S$
- $S, [S : P] : P$
- $P, P : S$
- $S, M : \text{ROOT}$
- $\text{ROOT}, B : B$

Rule 2.1 allows us to define the structure of a sentence in which all clauses are rhipidial. Let us call this type of sentence the rhipidial sentence.

Definition 2.2: Rhipidial sentence

A sentence whose semantic base is a linear structure  $\left[ [B, {}_R, [S : P]]^i, B, {}_R, [S : P]^i, M \right]$ , where a superscripted variable indicates the number of repetition and  $B$  may be a linear structure  $\left[ [B, {}_R, [S : P]]^j, B, {}_R, [S : P]^j, M, B \right]$ , to which the same rules of variables and  $B$  apply recursively, is a rhipidial sentence.

## References